

TEFILLAH

Complete System Guide and Hardening Manual

A walkthrough of the entire system from the inside out, written so a curious 12-year-old can follow it and a non-technical adult can rely on it. Plus a security hardening roadmap.

Audience

Curious 12-year-old, non-technical adult, future engineers maintaining this system.

Version

v1.0 - written the day tefillah.in went live on AWS.

Read with patience. Each chapter starts with a story for a child, then unfolds into what an engineer needs to know. Skip whichever part already feels obvious.

Table of Contents

Part 1 What Tefillah Is	3
Part 2 The Pieces, Explained Simply	5
Chapter 1 The Mobile App	5
Chapter 2 The New Web App (tefillah.in)	6
Chapter 3 The Admin Panel	8
Chapter 4 The Backend Brain	9
Chapter 5 The Database (MongoDB)	11
Chapter 6 The Address Book (DNS)	12
Chapter 7 All the Helpers (Email, Auth, etc.)	14
Part 3 How a Prayer Travels (Request Flow)	16
Part 4 The Tools and Services Inventory	18
Part 5 The AWS Deploy We Performed Today	21
Part 6 Security Hardening Roadmap	25
6.1 Honest Framing of 'Non-hackable'	25
6.2 Do TODAY (critical)	26
6.3 Do this WEEK (high priority)	28
6.4 Do this QUARTER (defence in depth)	30
6.5 Cost Summary	32
Part 7 Operations and Maintenance	33
Part 8 Glossary	35

Part 1 - What Tefillah Is

Story for a 12-year-old

Imagine a quiet church where anyone in the world can drop a folded note into a wooden box. A volunteer reads each note, prays over it, and leaves a small scripture verse on the windowsill in return. Tefillah is that church, but it lives on the internet. The wooden box is a phone app and a website. The volunteers are real people who signed up to be 'prayer partners.' The scripture verse comes back to the person who wrote the note. That's it.

For the grown-up

Tefillah is a multi-tenant prayer-request platform with three client surfaces (iOS/Android via Expo, a responsive web app, and an operator admin panel) sharing one FastAPI backend on AWS Elastic Beanstalk, a MongoDB Atlas database, and several third-party services for email, identity, and language generation. Users submit prayers; the system routes them to vetted partner volunteers based on region; partners acknowledge each prayer; the user receives a scripture verse in response.

Two kinds of people use the app:

- **Users** - any person who wants to submit a prayer, read scripture daily, or look back at their own prayer history.
- **Prayer Partners** - vetted volunteers who receive prayer requests, pray over them, and mark them complete. The admin must approve a partner before they can receive requests.
- **Admins** - the people who run the platform (you). They use the admin panel to onboard partners, watch usage, and handle reports.

Three places the same software lives:

- **Mobile app** for phones (will be in the Apple App Store and Google Play once you publish).
- **tefillah.in** for desktop and mobile browsers - this is the one we shipped today.
- **admin.tefillah.in** for operators - same code style as tefillah.in but with admin-only features.

All three talk to the *same backend* at **api.tefillah.in**. So when a user submits a prayer from their iPhone, the admin sees it instantly in the panel. There is one source of truth.

Part 2 - The Pieces, Explained Simply

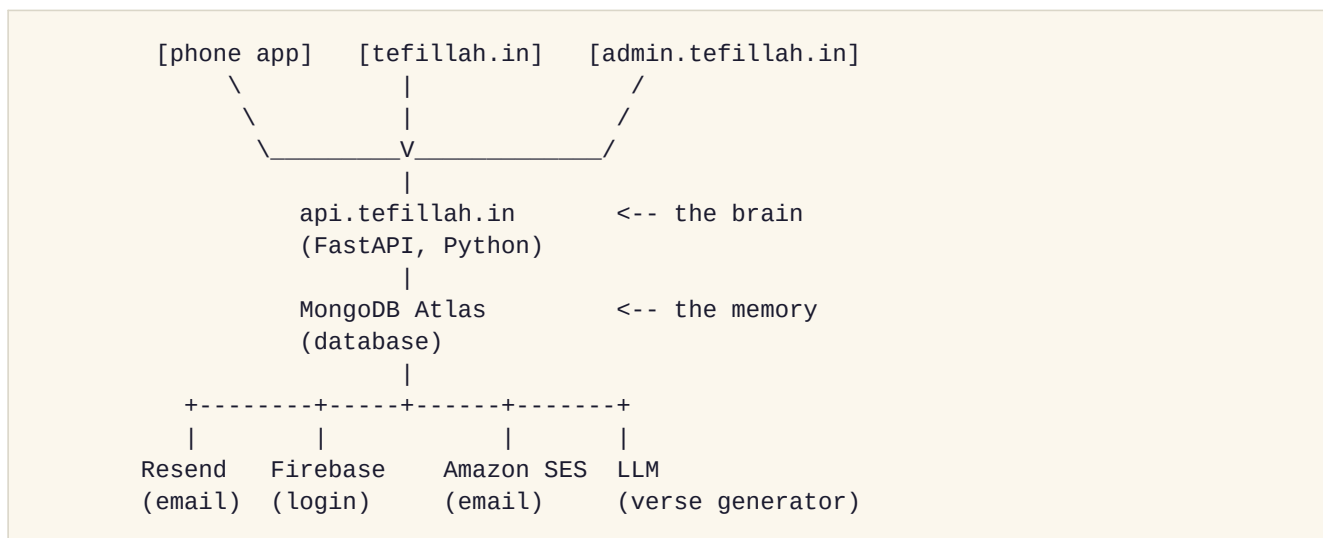


Figure: The whole system on one page. Everything routes through the backend.

Chapter 1 - The Mobile App

Story for a 12-year-old

The mobile app is like a small magical notebook in your pocket. You open it, write a prayer, and tap a button. The notebook quietly sends the prayer out into the world to find a partner who can pray for you. Then it shows you a scripture verse. The notebook also remembers every prayer you ever wrote in it, and lights up softly when someone has prayed over one of them.

Where the code lives:

- **frontend/** in the project repository.
- Built with **Expo** (a framework on top of React Native) so the same JavaScript code becomes a real iOS app and a real Android app.
- Screens are organised by folder: **(auth)/** for sign-in flows, **(main)/** for the after-login user experience, **(partner)/** for the partner dashboard.

What it stores on the phone:

- The login token (called a JWT - a small encoded string proving who you are) is stored in **expo-secure-store**, which is the phone's keychain on iOS and Keystore on Android. The app cannot share this with another app.
- Language preference and theme choice in regular storage.
- Nothing else. All real data lives on the server.

Notable libraries:

- **axios** - sends HTTP requests to the backend
- **zustand** - small state management library (keeps track of who is logged in)
- **expo-router** - turns files into navigable screens, like Next.js but for mobile

- **expo-notifications** - lets the app receive push notifications when a partner has prayed for you
- **i18next + react-i18next** - supports multiple languages
- **firebase** - allows 'sign in with Google'

Chapter 2 - The New Web App (tefillah.in)

Story for a 12-year-old

Imagine the church has a website now. The website looks elegant - cream and gold like the inside of an old book - and works exactly like the mobile app. You can sign up, write a prayer, see your history, everything. It looks classic but feels modern. You can use it on a phone browser too. It is fast because it lives on hundreds of servers across the world, and your computer talks to the closest one.

Where the code lives:

- **tefillah-web/** in the project repository.
- Built with **Vite** (a fast build tool) + **React 19** + **TypeScript**.
- Styled with **Tailwind CSS v4** (utility classes like text-2xl, mt-4, bg-gray-900).
- Pages live under **tefillah-web/src/pages/**. There are 18 pages.

Pages in the web app:

Public pages	Auth/recovery pages	User-only (after login)	Partner-only
LandingPage (the gold-on-cream hero)	LoginPage	HomePage (verse + quick actions)	PartnerDashboardPage
PrivacyPage	SignupPage	PrayerPage (submit form)	
TermsPage	VerifyPage (6-digit OTP)	ConfirmationPage (post-submit thank-you)	
NotFoundPage (gentle 404)	ForgotPasswordPage / Reset	HistoryPage (timeline of prayers)	
PartnerLoginPage	ChangePasswordPage	MenuPage (settings, sign out)	
PartnerSignupPage		NotificationsPage	

How it loads from the user's perspective:

- Browser requests `https://tefillah.in/`.
- DNS (Route 53) returns CloudFront edge IP.
- CloudFront serves **index.html** (about 1.3 KB).
- Browser reads the script tag, fetches the main JavaScript bundle (about 110 KB gzipped). CloudFront serves it from its cache, immutable for one year.
- JavaScript boots, React renders the landing page, and the page fetches a daily verse from **api.tefillah.in**.
- When the user clicks 'Sign in', React Router shows the LoginPage. No new page load - the URL changes but the same JavaScript handles it. That's what 'SPA' means - Single Page Application.

The breathing logo (a nice detail):

The flame icon at the top is the Ionicons 'flame' SVG, with two extra concentric circles behind it. Each layer breathes on its own CSS animation timeline: the outer circle slowly expands and dims; the inner circle pulses opacity; the flame gently scales up and down. The combined effect is that the logo visibly 'luminates' but never feels jittery.

Chapter 3 - The Admin Panel

Story for a 12-year-old

The admin panel is the private back-office where you, the operator, can see everyone using the church. You can approve new partners, look at the prayers that have been submitted (only when investigating a problem - normal operations don't read prayers), see who is currently online, and watch numbers grow.

- Lives in **admin-panel/** in the repository. Same tech stack as tefillah-web: Vite + React + TypeScript + Tailwind.
- Hosted on **Cloudflare Pages** at the address **tefillah-admin.pages.dev**. (Not on AWS - this is a leftover from before today's migration. It still works because DNS for admin.tefillah.in is just a CNAME pointing at Cloudflare Pages.)
- Talks to the same **api.tefillah.in** backend, but uses admin-only endpoints under **/api/admin/***. Those endpoints require a valid admin JWT and refuse anyone else.

Pages in the admin panel: Dashboard, Users, Partners, Prayers, Analytics, Calendar, Admins, Audit Logs, Notifications.

Chapter 4 - The Backend Brain

Story for a 12-year-old

The brain is a Python program that listens for messages from the phone apps and websites all day long. Every time someone signs up, sends a prayer, asks for a verse, or logs in, the brain reads the message, checks if it's allowed, talks to the database, sometimes asks a friend (the verse generator, the email sender) for help, and sends an answer back. It is one big Python file: **server.py**.

The technical stack:

Component	What it does
FastAPI (Python 3.12)	The web framework. Defines all the URL endpoints and their inputs/outputs.
Uvicorn	The actual HTTP server that runs the FastAPI code. Listens on port 8000.
Motor	Async driver for MongoDB - allows the backend to talk to the database without blocking other requests.
Pydantic	Validates that incoming JSON has the right shape. If someone sends bad data, FastAPI rejects with a 422 automatically.
PyJWT	Issues and verifies the JSON Web Tokens (login proofs).
bcrypt / passlib	Hashes passwords. The database NEVER stores a plaintext password.

Component	What it does
firebase-admin	Verifies Google sign-in tokens from the mobile app.
Resend SDK	Sends transactional email (signup confirmations, password resets).
httpx / requests	Used to call third-party APIs from inside the backend (the LLM, payment gateways if added).
python-dotenv	Loads secrets from .env in development. In production these come from EB environment variables.

Where it runs:

- AWS **Elastic Beanstalk** in the **ap-south-1** region (Mumbai). EB is Amazon's 'I'll run a web app for you' service: you upload a zip, EB installs dependencies, starts the server, and puts it behind a load balancer.
- The custom domain **api.tefillah.in** is a CNAME to the EB load balancer.
- Logs are visible in the EB web console under 'Logs'.

Endpoints, grouped by family:

URL group	Example endpoints	Purpose
/api/auth/*	register, login, verify-email, forgot-password, reset-password, change-password, me, social	User identity and account lifecycle
/api/prayer/*	submit, guest-submit, history	Prayer submission and reading history
/api/partner/*	register, login, stats, requests, requests/{id}/mark-prayed, profile	Partner onboarding and workflow
/api/user/*	notifications, notifications/{id}/read, read-all, register-device	Notification inbox and push-token registration
/api/admin/*	dashboard, users, partners, prayers, analytics, audit-logs	Admin panel
/api/verse/generate	(public, query param: language)	Calls the LLM to produce a daily verse. Falls back to fixed verses if LLM fails.
/api/health	(public)	Returns 200 OK. Used by monitors and load balancers.

Chapter 5 - The Database (MongoDB)

Story for a 12-year-old

The database is the brain's memory. It is a giant shelf of folders. Every time someone signs up, a new folder appears in the 'users' drawer. Every time someone writes a prayer, a folder appears in the 'prayers' drawer. The brain can open any folder by name and read what's inside. The database itself lives on a different company's computers (MongoDB Atlas) - not on your AWS account.

The eight collections (think of them as drawers or tables):

Collection	What's in it
users	Every user account: name, email (lowercased), bcrypt password hash, phone, city, is_verified, created_at.
partners	Every partner: same as users plus organization, partner_type, cell_id, total_prayer_time_minutes, prayers_handled, last_active (for the 'Online Now' counter).
prayers	Every prayer ever submitted: content, category, status (pending, assigned, prayed), assigned partner id, returned bible verse, comfort message, created_at.
notifications	Per-user/partner inbox of short messages and pings.
admin	Admin user accounts (separate from regular users for safety).
email_verifications	Short-lived 6-digit OTP codes for email confirmation. Auto-expire.
password_resets	Short-lived 6-digit codes for password recovery. Auto-expire.
rate_limits	Per-email throttling state. E.g. 'no more than 3 password reset attempts per hour'.

Why MongoDB and not SQL? - The data shapes vary (a prayer has different fields from a notification), and we don't do complex joins, so a document database is the simpler fit. The trade-off is that you cannot run easy 'SELECT name, count(*) FROM users JOIN prayers' style queries; you have to think in aggregation pipelines.

Chapter 6 - The Address Book (DNS)

Story for a 12-year-old

The internet is like a giant city. Every website has a number address (an IP address) like 192.168.1.1. Humans hate numbers, so we use names like tefillah.in. The DNS is the giant phonebook that turns names into numbers. When you type tefillah.in in a browser, the browser secretly asks the phonebook 'what's the number?' and only then knocks on the door. The phonebook for tefillah.in lives in AWS Route 53 now. Before today, it lived at Cloudflare.

The full picture today:

```

Visitor's computer asks: 'where is tefillah.in?'
      |
      V
ISP resolver --> root DNS --> .in TLD --> 'ask Route 53'
      |
      V
Route 53 hosted zone (Z06246302MVTLD5P84FBW)
  returns the right answer for the right name:

tefillah.in      A ALIAS    -> CloudFront edge
www.tefillah.in  A ALIAS    -> CloudFront edge
api.tefillah.in  CNAME      -> EB load balancer
admin.tefillah.in CNAME     -> tefillah-admin.pages.dev
MX records       -> Cloudflare Email Routing
TXT records      -> SPF, DKIM, DMARC

```

Figure: Route 53 is the new authoritative answer-keeper for tefillah.in.

Why the apex uses an ALIAS and not a CNAME:

DNS rules forbid CNAME at the apex (the root domain, with no subdomain). Route 53 offers a non-standard 'ALIAS' record that behaves like a CNAME but is allowed at the apex. It also has no extra DNS lookup cost. This is one of the small reasons people pick Route 53 over generic DNS providers for AWS deployments.

Chapter 7 - All the Helpers (Email, Auth, etc.)

The backend talks to several outside services. Each one does one job well. None of them sees your user's password.

Service	What it does	How it talks to us
Resend	Sends emails the backend wants to send out (verify your email, reset password).	Backend POSTs to api.resend.com with our API key in the Authorization header.
Amazon SES	Alternative outbound email path. The DNS has SPF/DKIM records pointing at SES, so SES email passes spam checks.	Backend uses the AWS SDK with the IAM user's keys.
Firebase	Lets users 'Sign in with Google'. Firebase verifies the Google identity and returns a token; our backend trusts Firebase to confirm 'yes, this is really suraj@gmail.com.'	Mobile app uses the Firebase client SDK; backend uses firebase-admin to verify the token signature.
LLM provider	Generates the daily verse and comfort messages.	Backend POSTs the prayer text and asks for a relevant verse. The response is returned to the app. Fall-back static verses kick in if the LLM is down.
Cloudflare Email Routing	Receives email sent TO @tefillah.in addresses. Configured to forward to a chosen inbox.	Email servers around the world deliver to Cloudflare's MX servers; Cloudflare looks up the routing rule for the recipient.

Part 3 - How a Prayer Travels (Request Flow)

Story for a 12-year-old

Let's follow Mary as she submits a prayer for her grandmother. Mary opens tefillah.in on her laptop. She types her prayer and clicks 'Lift this prayer.' Now imagine that prayer is a tiny letter being passed through a long chain of helpers. We're going to follow the letter from Mary's keyboard all the way to the database, and back.

Step by step (~10 hops):

1. Mary's browser puts her prayer text in a JSON object, attaches her login JWT (from localStorage), and sends POST to `https://api.tefillah.in/api/prayer/submit`
2. Her ISP's DNS asks Route 53 for `api.tefillah.in`. R53 returns the EB load balancer hostname.
3. Browser opens TLS to the EB load balancer. TLS is the lock that encrypts everything between Mary and the server.
4. EB load balancer picks a healthy backend instance and forwards the request.
5. The Python backend (FastAPI) parses the JSON. Pydantic confirms the shape is correct (content not empty, etc).
6. The backend verifies Mary's JWT signature with the `JWT_SECRET`. If valid, it extracts her user id from inside the token.
7. The backend asks MongoDB Atlas: 'is this user verified and not banned?' Atlas answers.
8. The backend records a new prayer document in MongoDB.
9. The backend asks the LLM for a verse appropriate to the prayer's category. If the LLM is down, it picks a random fallback verse.
10. The backend returns JSON: `prayer_id`, `category`, `comfort_message`, `bible_verse`, `bible_reference`.
11. Mary's browser receives the JSON, navigates her to the `/prayer/confirmation` page, and shows her the verse with the 'thank you' animation.

Figure: A prayer's journey from Mary's keyboard back to her screen.

Why HTTPS matters at every hop:

If any of these hops happened over plain HTTP, someone on Mary's WiFi network could read her prayer in transit. With HTTPS, even her own ISP cannot read it. Today, every hop in the chain above happens over TLS 1.2 or higher.

Where each piece of data ends up:

- Mary's prayer text -> MongoDB Atlas (encrypted at rest by Atlas).
- Mary's JWT -> stays in her browser's localStorage.
- Mary's email -> only known to MongoDB and Resend (when sending verification emails).
- Mary's IP address -> visible to CloudFront, the EB load balancer, and MongoDB Atlas (which logs connection metadata).
- Mary's password (only at signup or change) -> bcrypt-hashed and stored as a hash. The plaintext is never written to disk anywhere.

Part 4 - The Tools and Services Inventory

Every external account, framework, and tool involved. If you wanted to inventory the whole system for an audit, this is the list.

Programming languages

Language	Where	Why
TypeScript / JavaScript	All three frontends (mobile, web, admin)	Browser language; React's first-class language
Python 3.12	Backend (server.py)	FastAPI ecosystem, easy to write, fast enough for our scale
HTML / CSS / Tailwind	All UI	Standard web building blocks
MongoDB query language	Backend -> Atlas	Document-style queries, no SQL

Frameworks and libraries

Mobile (Expo 54, React Native 0.81):

- expo-router, expo-secure-store, expo-notifications, expo-location
- axios, zustand, i18next, react-i18next
- firebase JS SDK, @react-native-google-signin
- react-native-reanimated for animations

Web (Vite + React 19):

- react-router 7, axios, zustand
- Tailwind CSS v4 via @tailwindcss/vite
- lucide-react for icons
- Cormorant Garamond + Inter from Google Fonts

Backend (FastAPI on Python 3.12):

- fastapi 0.110, uvicorn 0.25
- motor 3.3 + pymongo 4.5 (MongoDB)
- pyjwt + bcrypt + passlib (auth)
- firebase-admin 6.4 (Google sign-in verification)
- httpx, requests (third-party API calls)
- python-dotenv (config)

AWS resources (account 020262236044)

Resource	Identifier	Region
IAM user (CLI deploy)	tefillah-web-deploy	global
Elastic Beanstalk environment	tefillah-api-prod (FastAPI backend)	ap-south-1

Resource	Identifier	Region
S3 bucket	tefillah-web-prod (private, OAC-only)	ap-south-1
CloudFront distribution	E20DJ1IDF5M5MD (dc71sb5z88jh2.cloudfront.net)	global
CloudFront OAC	E3EQPRBAT2OUWZ	global
CloudFront response headers policy	3c3cd71b-468f-4ef7-bfbf-4efb649a29bf	global
ACM certificate	e1749f45-... (tefillah.in + www.tefillah.in)	us-east-1
Route 53 hosted zone	Z06246302MVTLD5P84FBW (tefillah.in)	global
Amazon SES	(used for outbound email per DNS SPF/DKIM)	ap-northeast-1

Third-party vendors

Vendor	Purpose	Account login
MongoDB Atlas	Hosted MongoDB cluster (the database)	Your Atlas account email
Cloudflare	Hosts admin.tefillah.in on Pages; was the DNS provider until today	tefillahprayerapp@gmail.com
GoDaddy	Domain registrar - owns tefillah.in	Your GoDaddy account
Resend	Outbound transactional email	Your Resend account
Firebase	Google sign-in + push notifications	Your Google account
LLM provider (OpenAI/Anthropic)	Generates daily verses and comfort messages	Your account at the provider
Expo / EAS	Builds the mobile app for the stores	Your Expo account

Where every secret lives on disk

File	What's inside	Sensitivity
C:\Users\suraj\.aws\credentials	AWS access key ID + secret for tefillah-web-deploy	HIGH - never share, never commit
backend\.env	MongoDB URI, JWT_SECRET, Resend API key, Firebase Web API key, Google client ID, FIREBASE_PROJECT_ID	HIGH - never commit
backend\firebase-credentials.json	Firebase service-account JSON (admin-level for the Firebase project)	HIGH - never commit
tefillah-web\.env	VITE_API_URL only (public URL)	LOW

Part 5 - The AWS Deploy We Performed Today

This is a faithful log of what happened today. If you ever need to repeat it for a fresh environment, follow this sequence.

Step 1 - Inspect the current Cloudflare zone

I called the Cloudflare dashboard API using your browser session cookies and pulled all 17 existing DNS records (CNAMEs, MX, TXT, SRV) for tefillah.in into a JSON file in your Downloads folder.

Step 2 - Create the Route 53 hosted zone

```
aws route53 create-hosted-zone --name tefillah.in \
--caller-reference tefillah-r53-... \
--hosted-zone-config 'Comment=Tefillah primary zone'
```

AWS returned the zone ID and four nameservers. Those four NS values are what GoDaddy needs.

Step 3 - Mirror every Cloudflare record into Route 53

A Python script (import-cf-to-r53.py) parsed the JSON, grouped records by (name, type), translated each into a Route 53 change-batch entry, and applied them with change-resource-record-sets. The apex A records and www CNAME were skipped because they would be replaced later with ALIAS records pointing at CloudFront.

Step 4 - Build the production assets

```
cd tefillah-web
npm install
npm run build # -> dist/ folder with index.html + assets/
```

Step 5 - Create the S3 bucket (private)

```
aws s3api create-bucket --bucket tefillah-web-prod \
--region ap-south-1 \
--create-bucket-configuration LocationConstraint=ap-south-1

aws s3api put-public-access-block --bucket tefillah-web-prod \
--public-access-block-configuration \
'BlockPublicAcls=true,IgnorePublicAcls=true,\
BlockPublicPolicy=true,RestrictPublicBuckets=true'

aws s3api put-bucket-versioning --bucket tefillah-web-prod \
--versioning-configuration Status=Enabled
```

Step 6 - Upload the assets with smart cache headers

```
# Hashed assets get year-long immutable caching
aws s3 sync dist/assets s3://tefillah-web-prod/assets \
--cache-control 'public, max-age=31536000, immutable'

# index.html, icons get no-cache (so updates show up instantly)
aws s3 sync dist s3://tefillah-web-prod \
--cache-control 'no-cache, must-revalidate' \
--exclude 'assets/*' --delete
```

Step 7 - Request ACM certificate (us-east-1)

```
aws acm request-certificate \
--region us-east-1 \
--domain-name tefillah.in \
--subject-alternative-names www.tefillah.in \
```

```
--validation-method DNS \  
--key-algorithm RSA_2048
```

ACM returns two DNS CNAME values to prove ownership. Those go into Route 53.

Step 8 - CloudFront response headers policy

A policy named 'tefillah-web-security-headers' was created with: HSTS (max-age 1 year, includeSubDomains, preload), CSP locked to api.tefillah.in for connect-src, X-Frame-Options DENY, X-Content-Type-Options nosniff, Referrer-Policy strict-origin-when-cross-origin, Permissions-Policy disabling camera/microphone and restricting geolocation to first-party only.

Step 9 - CloudFront Origin Access Control + Distribution

```
# OAC so the distribution can sign requests to S3  
aws cloudfront create-origin-access-control \  
--origin-access-control-config file://oac.json  
  
# Distribution (initial - no aliases yet)  
aws cloudfront create-distribution \  
--distribution-config file://cloudfront-distribution-initial.json
```

Distribution domain: dc71sb5z88jh2.cloudfront.net. SPA rewrite is enabled by mapping 403/404 to /index.html with response code 200.

Step 10 - Bucket policy scoped to the new distribution only

```
{  
  "Version": "2012-10-17",  
  "Statement": [{  
    "Sid": "AllowCloudFrontReadOnly",  
    "Effect": "Allow",  
    "Principal": {"Service": "cloudfront.amazonaws.com"},  
    "Action": "s3:GetObject",  
    "Resource": "arn:aws:s3:::tefillah-web-prod/*",  
    "Condition": {"StringEquals": {  
      "AWS:SourceArn": "arn:aws:cloudfront::020262236044:distribution/E20DJ1IDF5M5MD"  
    }}  
  ]  
}
```

This is the magic that makes the bucket private while still serving traffic globally.

Step 11 - Change nameservers at GoDaddy (the only manual step)

- Logged in to GoDaddy, opened tefillah.in.
- Set custom nameservers to: ns-258.awsdns-32.com, ns-934.awsdns-52.net, ns-1137.awsdns-14.org, ns-1774.awsdns-29.co.uk.

Step 12 - Wait for global propagation

A Python script polled Google's and Cloudflare's DNS-over-HTTPS resolvers every 90 seconds. When both confirmed AWS nameservers, the script moved on. In our case, this took about 40 minutes.

Step 13 - ACM cert auto-validates, distribution rebound

ACM checks the two validation CNAMEs in Route 53. Once visible, cert flips to ISSUED. Distribution updated to add tefillah.in + www.tefillah.in aliases and bind the ACM certificate. Minimum TLS bumped to 1.2_2021.

Step 14 - Add the two A-ALIAS records and run live smoke tests

R53 A ALIAS records for the apex and www pointing at the CloudFront distribution. After ~30 seconds for CloudFront to redeploy, <https://tefillah.in/> returned HTTP 200 with the full security header set, the SPA rewrite worked for deep links, and HTTP redirected to HTTPS.

What you can re-run later

Any future website deploy is now this short sequence:

```
cd tefillah-web
npm run build
aws s3 sync dist/ s3://tefillah-web-prod/ --delete
aws cloudfront create-invalidation --distribution-id E20DJ1IDF5M5MD --paths "/*"
```

Part 6 - Security Hardening Roadmap

6.1 Honest framing of 'non-hackable'

Read this first

There is no such thing as a system that is literally non-hackable. Anything connected to the public internet can, in principle, be attacked. What we can do is make a successful attack require sophisticated tools, significant time, and luck. The recommendations below raise the cost of attack and the speed of detection. If someone with a nation-state budget targets you specifically, no amount of off-the-shelf hardening will hold. But everyday opportunistic attacks, credential stuffing, and accidental data leaks are absolutely preventable with what's listed here.

6.2 Do TODAY (critical)

Action 1 - Rotate JWT_SECRET

Why this is critical

The current value in backend/.env is literally the placeholder text that includes the word 'change-in-production'. JWT_SECRET is what the backend uses to sign every login token. If anyone obtains this string, they can forge an admin token and access everything as you. Treat it like a master password.

- Generate a new random value: `python -c "import secrets; print(secrets.token_urlsafe(48))"`
- Update backend/.env with the new value (local development).
- In the AWS EB console, open the environment -> Configuration -> Software -> set JWT_SECRET environment variable to the new value.
- EB will restart the application. Every existing user session becomes invalid (everyone has to log in again). This is a one-time cost, not a recurring one.
- Cost: \$0. Time: 5 minutes.

Action 2 - Confirm backend/.env is not in git

- Run `cd backend && git check-ignore -v .env`. It must report .env is ignored.
- Also confirm `firebase-credentials.json` is in .gitignore.
- If they were ever committed historically, rotate every secret in them (MongoDB password, Resend API key, Firebase service account, LLM key).
- Cost: \$0. Time: 5 minutes (to check) or up to 2 hours (if rotation is needed).

Action 3 - Enable MFA on every admin-grade account

- AWS root user (most important): authenticator app or hardware key (YubiKey).
- AWS IAM users with write permissions (tefillah-web-deploy if you grant it console access later).
- Cloudflare (still hosts admin.tefillah.in Pages project + email routing).

- GoDaddy (anyone who controls the domain registrar controls everything).
- MongoDB Atlas (database access).
- Firebase (controls Google sign-in for the mobile app).
- Gmail account tied to all the above.
- Cost: \$0 (authenticator app) to \$50 once (YubiKey). Time: 30 minutes.

Action 4 - Move secrets out of .env into AWS Systems Manager Parameter Store

- Why: secrets in environment variables are visible to anyone who can read the EB config or SSH onto the instance. Parameter Store with SecureString encryption is the proper place.
- Migrate one at a time: `aws ssm put-parameter --name /tefillah/prod/JWT_SECRET --type SecureString --value '...'`
- Update the backend to read these at startup via boto3.
- Remove the values from EB environment variables once parameters are loading correctly.
- Cost: \$0 (Parameter Store standard tier is free up to 10k parameters). Time: 2 hours.

6.3 Do this WEEK (high priority)

Action 5 - Tighten the tefillah-web-deploy IAM policy

- Today this user has 4 *FullAccess policies. Replace with a tight inline policy scoped to: only the specific S3 bucket, only the specific CloudFront distribution, only the specific Route 53 zone, only the specific ACM cert.
- If the keys ever leak, the blast radius is limited to those resources, not the entire account.
- Cost: \$0. Time: 1 hour to write and test the policy.

Action 6 - Enable AWS GuardDuty + CloudTrail across the account

- CloudTrail records every API call to AWS. GuardDuty watches CloudTrail for suspicious patterns (someone trying to disable logging, IAM keys being used from an unusual country, instances making cryptocurrency-mining DNS queries).
- Both can be turned on with a few clicks in the console.
- Cost: ~\$3-15/month depending on activity volume. Time: 15 minutes.

Action 7 - Lock MongoDB Atlas to your specific origins

- In Atlas, set the IP allowlist to only the EB instance public IPs (use a VPC peering or PrivateLink for the strongest version).
- Database user permissions: create a dedicated user for the backend with only the collections it needs (not 'root@admin').
- Enable Atlas continuous backup (point-in-time recovery).
- Cost: Atlas backup is included in M10+ tiers; PrivateLink adds ~\$30/month per region. Time: 1 hour.

Action 8 - Deploy AWS WAF in front of CloudFront

- Adds a managed ruleset (AWSManagedRulesCommonRuleSet, AWSManagedRulesKnownBadInputs, AWSManagedRulesAmazonIpReputationList) that blocks

obvious junk traffic before it hits the backend.

- Plus rate-based rules: 'block any IP making more than 500 requests/5 minutes.'
- Cost: ~\$5-15/month at low traffic. Time: 30 minutes.

Action 9 - Enable AWS Shield Standard (free) and verify it's protecting CloudFront

- Shield Standard is automatically on for all CloudFront distributions. Verify in the console.
- Shield Advanced is \$3000/month and overkill unless you become a public target.
- Cost: \$0. Time: 5 minutes.

Action 10 - Backend rate limiting per IP, not just per email

- Current backend throttles by email (3 password-reset attempts per hour). An attacker rotating emails bypasses this.
- Add IP-based throttling using slowapi (FastAPI extension): 60 requests/min for /api/auth/login, 5 requests/min for /api/auth/forgot-password.
- Cost: \$0. Time: 2 hours.

Action 11 - Subscribe to dependency-vulnerability alerts

- Enable GitHub Dependabot (free) on the repo. It opens PRs when one of your npm or pip dependencies gets a published CVE.
- Add Snyk free tier or npm audit --production to CI for the same purpose.
- Cost: \$0 for the free tier. Time: 30 minutes.

6.4 Do this QUARTER (defence in depth)

Action 12 - Centralised logging and alerting

- Route EB application logs to CloudWatch Logs, then set up CloudWatch Alarms on key signals: 5xx error rate over 1%, P99 latency above 2s, sudden drop in /api/health 200s.
- Pair with SNS to send SMS or email alerts to you.
- Cost: \$5-20/month. Time: half a day.

Action 13 - Application-level error tracking (Sentry)

- Both the frontends and backend send caught exceptions to Sentry. You see real-time stack traces from production.
- Free tier covers ~5k events/month, plenty for a starting project.
- Cost: \$0 for free tier; ~\$26/month for the team tier.

Action 14 - Tighten Content Security Policy further

- Move from 'unsafe-inline' for styles to nonce-based CSP for stylesheets if you can do without inline styles (Tailwind's emit can be configured for this).
- Add a CSP report-uri so violations get reported to a tracker (Sentry supports this).
- Cost: \$0. Time: half a day if Tailwind cooperates.

Action 15 - Penetration test by a third party

- Before scaling to a large user base, commission a one-time pen test from a reputable firm like NCC Group, Cure53, or Trail of Bits, or a freelance via HackerOne.
- They will try to break in legitimately and write you a report.
- Cost: \$3000-15000 once. Time: 2-4 weeks calendar time.

Action 16 - Disaster recovery and backup verification

- Atlas continuous backup is good but verify your restore actually works. Do one restore-to-staging dry-run per quarter.
- Document the recovery steps in a runbook.
- Cost: \$0 if you already have Atlas backup. Time: half a day per dry-run.

Action 17 - Compliance baseline if you handle minors' data

- If users include children under 13 (US COPPA), under 16 (EU GDPR), have a privacy policy reviewed by counsel.
- Make data export and deletion endpoints in the admin panel.
- Cost: legal review ~\$500-2000.

6.5 Cost summary at a glance

Item	One-time cost	Monthly cost
JWT secret rotation	\$0	\$0
MFA / YubiKey	\$0-50	\$0
Parameter Store migration	\$0	\$0
IAM tightening	\$0	\$0
GuardDuty + CloudTrail	\$0	\$3-15
MongoDB Atlas PrivateLink (optional)	\$0	\$30
AWS WAF (managed rules)	\$0	\$5-15
AWS Shield Standard	\$0	\$0
Backend rate limiting (code)	\$0	\$0
Dependabot (GitHub)	\$0	\$0
CloudWatch alarms + SNS	\$0	\$5-20
Sentry (team tier)	\$0	\$0-26
Penetration test (yearly)	\$3000-15000	(amortised ~\$300-1250)

Total recurring at modest scale: roughly \$40-80 per month after the suggestions above. Adds up to a system that resists almost every opportunistic attack and recovers gracefully from accidents.

Part 7 - Operations and Maintenance

Daily web-app deploy (after a code change)

```
cd tefillah-web
npm install # only when dependencies change
npm run build # produces dist/
aws s3 sync dist/ s3://tefillah-web-prod/ --delete
aws cloudfront create-invalidation \
--distribution-id E20DJ1IDF5M5MD \
--paths '/index.html' '/'
# Asset files have hashed names so they don't need invalidation.
```

Backend deploy (after a code change to server.py)

```
cd backend
# Build a fresh deployment bundle
zip -r ../tefillah-api-v<n>.zip . -x 'venv/*' '__pycache__/*' '.env'
# Upload via EB CLI or AWS console -> Upload and Deploy
eb deploy tefillah-api-prod
```

Watch the EB Events tab while it deploys. Health goes Severe -> Warning -> Ok as instances cycle.

Cert renewal

Nothing to do. ACM automatically reaches out to Route 53 every 11 months and renews. As long as the validation CNAMEs we put in step 13 of the deploy stay in Route 53 (they're permanent), renewal is fully automatic.

Monitoring the live site

- Manual: `curl -sI https://tefillah.in/` - should return 200 and the security header set.
- Manual: `curl -s https://api.tefillah.in/api/health` - should return JSON.
- Automated: set up a free UptimeRobot or BetterUptime monitor on both URLs.

Rolling back a bad deploy

- Web app: S3 versioning is on. Restore the previous version of index.html and the previous assets/.
- CloudFront: not strictly needed for asset rollback (cache invalidation alone won't suffice).
- Backend: EB keeps the last N application versions. From the EB console, pick the previous version and 'Deploy.' Rollback takes ~3 minutes.

Adding a new admin user

- There is no public sign-up for admins by design. To create one, an existing admin must invite via the admin panel or you can insert directly into the MongoDB admin collection.
- Use bcrypt to hash the new admin's password before inserting.

Reading a user's prayer history (only when investigating)

Operators should only read prayers when responding to a specific complaint or abuse report. Every such read happens via the admin panel and is recorded in the `audit_logs` collection. The privacy promise in your Privacy Policy depends on this discipline.

Part 8 - Glossary

ACM - AWS Certificate Manager. Issues free SSL/TLS certificates and renews them automatically.

ALB / Load Balancer - Amazon Application Load Balancer. Spreads incoming HTTP requests across several backend servers and removes unhealthy ones.

ALIAS record - Route-53-specific record type that behaves like a CNAME but is allowed at the apex of a domain. Used to point tefillah.in directly at CloudFront.

API - Application Programming Interface. The set of URLs the backend exposes.

axios - JavaScript library used by both frontends to make HTTP requests.

bcrypt - Password-hashing algorithm. Slow on purpose, to make brute-force guessing impractical.

Cache-Control - HTTP header that tells browsers and CDNs how long to keep a copy.

CDN - Content Delivery Network. Geographically distributed servers that cache content close to users.

CloudFront - AWS's CDN service, sitting in front of our S3 bucket.

CNAME - DNS record that says 'this name is an alias for another name.'

CORS - Cross-Origin Resource Sharing. Browser security mechanism that decides whether code on one origin can call APIs on another.

CSP - Content Security Policy. HTTP header that tells the browser what scripts/styles/connections it is allowed to load.

DKIM - DomainKeys Identified Mail. Cryptographic signature on outbound email proving it came from your domain.

DMARC - Domain-based Message Authentication, Reporting and Conformance. Tells receivers what to do with email that fails SPF/DKIM checks.

DNS - Domain Name System. The phonebook that turns names like tefillah.in into IP addresses.

Elastic Beanstalk (EB) - AWS's 'we'll run your web app' service. Hides the EC2 instances, ALB, auto-scaling, etc.

Expo - Toolkit for building React Native apps with less ceremony. We use it for the mobile app.

FastAPI - The Python web framework the backend is written in.

Firebase - Google service. We use it for 'Sign in with Google' on the mobile app.

GoDaddy - The domain registrar where tefillah.in was bought.

HSTS - HTTP Strict Transport Security. Header that tells browsers 'always use HTTPS for this domain, never HTTP, ever, period.'

IAM - AWS Identity and Access Management. Where users, roles, and permissions live.

JWT - JSON Web Token. A short signed string that proves who you are after you log in.

LLM - Large Language Model. The AI service the backend uses to generate scripture verses and comfort messages.

MongoDB Atlas - MongoDB Inc's hosted database service - not in your AWS account.

MX record - DNS record that tells the world where to deliver email for a domain.

OAC - Origin Access Control. Newer mechanism for letting CloudFront read from a private S3 bucket.

OAI - Origin Access Identity. The older mechanism, deprecated in favour of OAC.

R53 / Route 53 - AWS's DNS service. Holds the authoritative records for tefillah.in.

Resend - Third-party transactional email service.

S3 - Simple Storage Service. AWS's hard drive in the cloud.

SES - Simple Email Service. AWS's outbound email service.

SPA - Single Page Application. A site where one HTML page is loaded and JavaScript switches between 'pages' without a full reload.

SPF - Sender Policy Framework. DNS TXT record listing which servers are allowed to send mail for the domain.

TLS - Transport Layer Security. The encryption inside HTTPS that prevents eavesdropping.

TXT record - Free-form DNS record. Used for SPF, DKIM, DMARC, and domain ownership proofs.

Uvicorn - Async Python HTTP server. Runs the FastAPI app inside Elastic Beanstalk.

Vite - JavaScript build tool. Used for tefillah-web.

WAF - Web Application Firewall. Filters bad requests before they reach your backend.

zustand - Small JavaScript state-management library used in both frontends.

End of guide. Treat this file as living documentation - update the relevant chapter whenever the system changes.